

M4.1 Atributos e métodos

Uma classe define estado e comportamento de um Objeto geralmente implementando métodos e atributos (nomes utilizados na maioria das linguagens modernas).

Os atributos, também chamados de campos (do inglês fields), indicam as possíveis informações armazenadas por um objeto de uma classe, representando o estado de cada objeto.

Os métodos são procedimentos que formam os comportamentos e serviços oferecidos por objetos de uma classe.

Outros possíveis membros de uma classe são:

- Construtores - definem o comportamento no momento da criação de um objeto de uma classe.
- Destrutor - define o comportamento no momento da destruição do objeto de uma classe. Normalmente, como em C++, é utilizado para liberar recursos do sistema (como memória).
- Propriedades - define o acesso a um estado do objeto.
- Eventos - define um ponto em que o objeto pode chamar outros procedimentos de acordo com seu comportamento e estado interno.

Atributos

Uma classe define os atributos e comportamentos comuns compartilhados por um tipo de objeto. Os objetos de certo tipo ou classificação compartilham os mesmos comportamentos e atributos. As classes atuam de forma muito parecida com um cortador de molde ou biscoito, no sentido de que você usa uma classe para criar ou instanciar objetos.

[...]

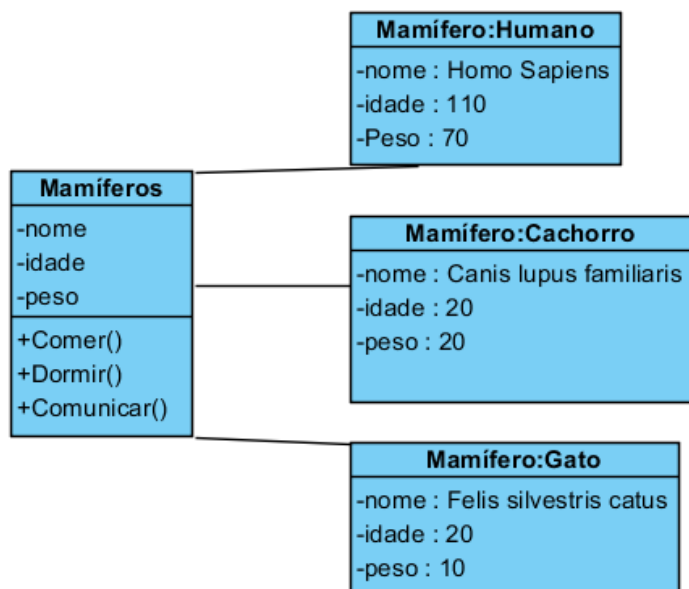
Atributos são as características de uma classe visíveis externamente. A cor dos olhos e a cor dos cabelos são exemplos de atributos (SINTES, 2002, p. 8).

Uma vez definida a classe, devemos iniciar o processo de abstrair os atributos de cada classe, ou seja, suas características e propriedades.

Assim como na classe, os nomes dos atributos não podem se repetir dentro da classe, ou seja, um mesmo nome de atributo pode ser utilizado em classes diferentes, mas nunca dentro de uma mesma classe.

Quando temos mais de um objeto, é importante observar a relação entre eles. Esses objetos podem existir independentemente entre si e se precisarem interagir, o farão por meio de mensagens. São as mensagens que fazem com que outro objeto realize alguma operação. Um segundo e importante modo de interação ocorre quando os objetos estão profundamente ligados entre si, de forma que o comportamento de um objeto depende do comportamento de outro. Um objeto pode conter outro dentro de si ou viver em simbiose entre si.

Podemos continuar nosso trabalho de abstração de dados por um aspecto mais biológico. Veja a seguir um exemplo:



Recomenda-se que o nome do atributo seja um substantivo que representa a propriedade do atributo. Por padrão, também é recomendado que o nome seja escrito em minúsculo e, em caso de nomes compostos, a primeira letra a partir da segunda palavra, seja escrita em maiúsculo. Exemplo: nomeFavorecido.

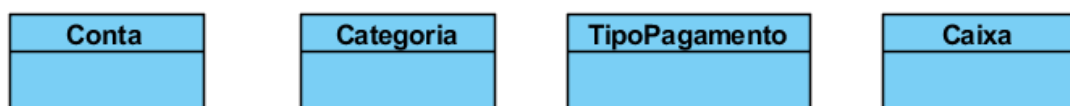
Para exemplificar vamos analisar o caso de um controle financeiro:

definimos nosso escopo para gerar um controle financeiro para nossa família. Portanto, agora temos condições de iniciar um processo para estreito de abstração para definirmos quem serão nossas classes.

Analisando o escopo definido podemos encontrar as seguintes classes:

- Conta: São os dados das minhas contas a pagar ou a receber.
- Categoria: É tipo de conta, ou seja, se são contas de telecomunicação, combustível, salário, dividendos, etc...
- Tipo de pagamento: Modalidade de pagamento, tais como dinheiro, cartão, depósito, etc...
- Caixa: Lugares de onde foi usada a modalidade de pagamento. Pode ser uma carteira ou uma conta do banco.

Assim as classes podem ser representadas:



Percebe-se que, por exemplo, o atributo nome é utilizado em todas as classes. Porém cada classe tem o seu atributo nome, ou seja, o mesmo atributo não está sendo compartilhado entre as classes. Existe o nome da Conta, o nome da Categoria, o nome do TipoPagamento e o nome do Caixa.

Esses atributos correspondem às características necessárias dentro do escopo de cada classe, portanto, é perfeitamente possível que algumas classes inúmeros atributos e outras classes tenham apenas um atributo. A necessidade de mais ou menos atributos em cada classe vai depender da utilização e da integração da classe em relação ao projeto.

Um grande erro que pode ocorrer neste estágio é a redundância de atributos. Redundância de atributos significa a existência da mesma informação sendo utilizada por diferentes classes de maneira não compartilhada.

É recomendado realizar, desde o início do projeto mais simples, uma simples documentação sobre suas classes. Como o escopo deste material não é o gerenciamento do projeto será utilizado aqui uma forma bem simples, porém eficiente, forma de documentar sua classe (Tabela 15).

Inserindo os atributos temos:

Conta
+nome
+valor
+dataVencimento
+dataPagamento

Categoria
+nome
+tipo

TipoPagamento
+nome

Caixa
+nome
+saldo

Descritivo dos atributos em cada classe

Classe	Atributo	Descrição
Conta	Nome	Nome da conta. Usado para descrever o conta em si.
	Valor	Valor da conta.
	dataVencimento	Data de vencimento da conta.
	dataPagamento	Data do pagamento efetivo da conta.
Categoria	Nome	Nome da categoria.
	Tipo	Tipo da categoria, pode ser Crédito ou Débito.
TipoPagamento	Nome	Nome da modalidade de pagamento. Exemplo: Dinheiro, Cheque, Cartão, Depósito.
Caixa	Nome	Nome do caixa.
	Saldo	Valor disponível no Caixa.

Uma vez definido os atributos da classe, é necessário definir também o tipo de dado utilizado em cada atributo.

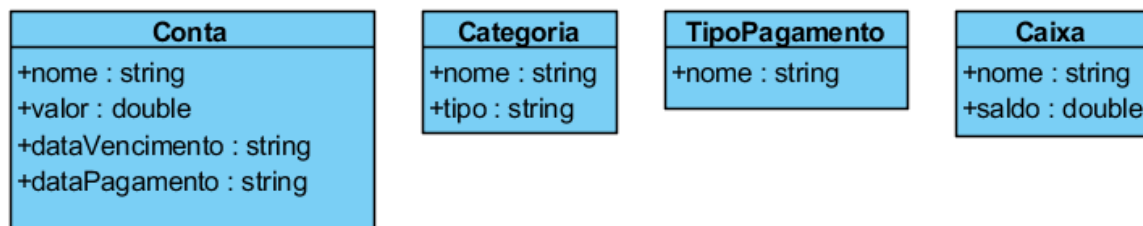
O tipo de dados utilizado em cada atributo assemelha-se muito à definição de tipos de dados utilizados em variáveis ou até mesmo dos campos de tabelas em bancos de dados. Desta forma, poderíamos dizer que teríamos uma limitação técnica em relação ao tipo de linguagem de programação seria utilizada para a implementação do *software* desenvolvido com POO. Porém, neste momento podemos utilizar o conceito de abstração (e para fins didáticos) para definir os tipos de dados de nossos atributos conforme Tabela a seguir.

Tipos de dados utilizados nos atributos

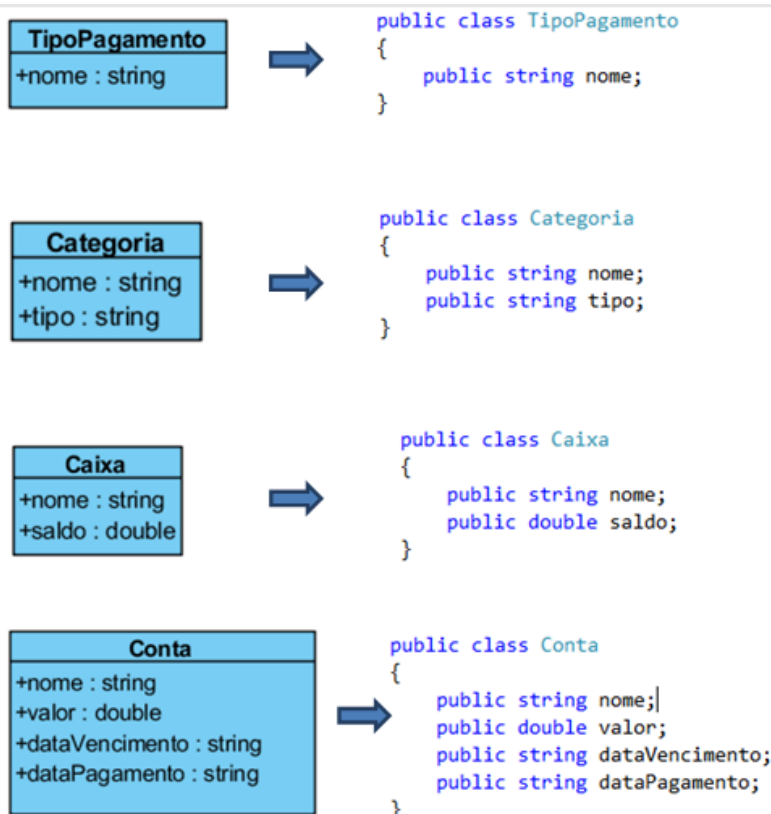
Nome	Descrição
Int	Tipo numérico inteiro com sinal de negativo.
Double	Tipo numérico real com sinal de negativo.
Bool	Tipo lógico. Armazena verdadeiro (True) ou falso (false).
String	Tipo texto. Armazena uma cadeia de caracteres.

DateTime	Tipo data e hora.
----------	-------------------

Desta maneira, seguindo as especificações e necessidades de cada atributo de nossas classes, temos os seguintes diagramas



assim temos os seguintes códigos:



Todos os atributos de uma classe podem ser de livre acesso ou não, ou seja, os atributos podem ser públicos ou privados. Esta restrição de acesso é chamada de qualificadores de acesso. Atributos públicos são aqueles atributos que podem ser vistos e acessados (manipulados) tanto pela classe a quem pertencem como por outras classes. Os métodos privados são vistos e acessados (manipulados) unicamente pela classe a que pertencem.

Iremos trabalhar melhor o conceito de atributos públicos e privados posteriormente em encapsulamento.

Métodos

Dando continuidade ao trabalho de abstração, abordaremos o conceito de métodos. Um método corresponde a uma ação (operação ou comportamento) que uma classe pode realizar.

Cada método funciona de forma independente, sendo utilizado apenas quando a ação é solicitada.

Todo o método é criado seguindo uma sintaxe (regras), sendo:

- As tarefas que um objeto pode realizar são definidas pelos seus métodos.
- Os Métodos são os elementos básicos para a construção dos programas OO
- Métodos não podem ser criados dentro de outros métodos ou fora de uma classe

Chamada do método:

nomeDaInstancia.nomeDoMetodo(parâmetros)

Todo o método é dividido em três partes:

- Nome do método.
- Argumentos (ou parâmetros).
- Retorno.

Todo o método deve ter um nome coerente com sua função.

Recomenda-se que o nome de cada método seja escrito com a primeira letra maiúscula e as demais letras minúsculas. Em caso de nomes compostos, a primeira letra de cada palavra inicia-se maiúscula e as demais letras minúsculas. Ex: RetornaSaldo().

- Os argumentos são declarados entre parênteses.
- Os métodos podem ter quantos argumentos forem necessários, não sendo obrigatório ter argumentos.
- Os métodos podem retornar alguma resposta em relação à ação realizada, porém não é obrigatório retornar algum valor.
- Assim como nos atributos, métodos podem ser públicos ou privados.

Os métodos são, na prática, onde será realizada a programação em si, ou seja, onde a inteligência da classe será definida e onde serão executadas as funções que a classe se propõe a realizar.

Portanto, todos os conceitos da programação estruturada se aplicam na programação dos métodos das classes, ou seja, variáveis, desvios condicionais, operações matemáticas e laços (loop) serão utilizados.

Métodos em C#

Os Métodos são definidos dentro de uma classe. Durante a execução de um programa, um método pertencente a um objeto desde que declaradas públicas podem ser acessadas pela referência da instância e o nome do método (Figura 48).

Diagrama de uma chamada de método em C#:

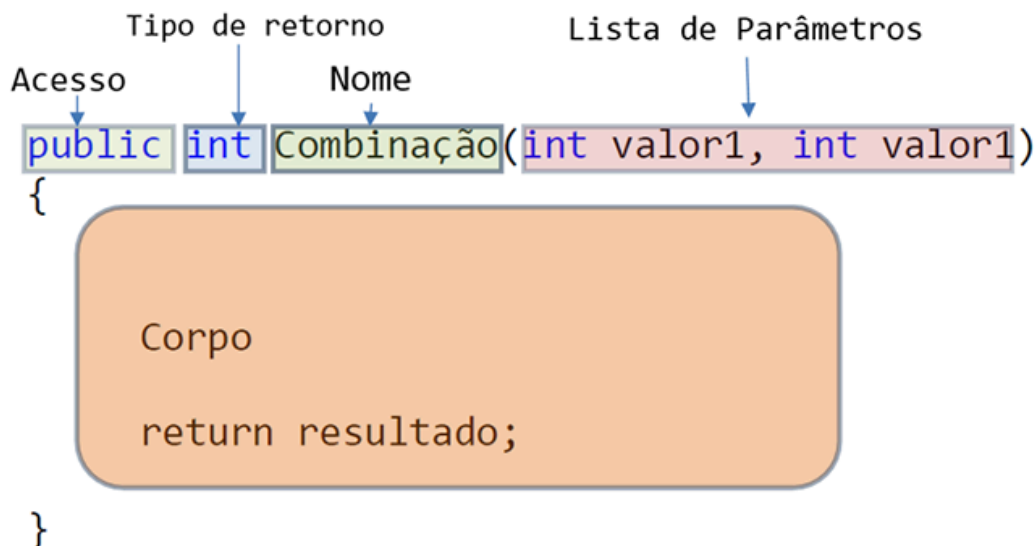
retorno Instância Nome do método Parâmetro

saldo = conta1.Saque(1000.00);

Forma geral da chamada de método

Para a construção dos métodos dentro de uma classe, os seus nomes devem ser iniciados com letras (incluindo os símbolos _ e \$), compostos de uma ou mais palavras sem espaço entre elas, podendo conter números. Os nomes dos métodos refletem ações que são efetuadas nos campos da classe e/ou valores passados como argumentos para esses métodos. Métodos não podem ser criados dentro de outros métodos ou fora de uma classe. Elas seguem a forma mostrada na anteriormente.

Para a construção dos métodos dentro de uma classe, os seus nomes devem ser iniciados com letras (incluindo os símbolos _ e \$), compostos de uma ou mais palavras sem espaço entre elas, podendo conter números. Os nomes dos métodos refletem ações que são efetuadas nos campos da classe e/ou valores passados como argumentos para esses métodos. Métodos não podem ser criados dentro de outros métodos ou fora de uma classe. Elas seguem a forma mostrada na figura a seguir:



O Nome é utilizado para chamar o método. Na linguagem C#, é uma boa prática definir os nomes dos métodos utilizando a convenção "Camel Case" com a primeira letra maiúscula. A lista de parâmetros define os valores que o método deve receber. Métodos que não precisam receber nenhum valor ficam com a lista de parâmetros vazia. No corpo fica a programação estruturada que define o que acontecerá quando o método for chamado. Finalmente o retorno é o tipo de dado que será devolvida como resposta ao final do processamento do método. Quando um método não devolve nenhuma resposta, ele deve ser marcado com a palavra reservada *void*.

Tomemos uma classe que é a conta corrente de um cliente, com os métodos para sacar e depositar:



Classe Conta corrente com os métodos depositar e Sacar

Na classe ContaCorrente no C# digitamos os métodos, depositar e sacar acumulando e retirando respectivamente do valor do Saldo

```
public class ContaCorrente
{
    public int numero;
    public String cliente;
    public float saldo;

    public void depositar(float valor)
    {
        saldo += valor;
    }
    public void sacar(float valor)
    {
        saldo -= valor;
    }
}
```

Inserindo métodos na classe ContaCorrente

Notamos que em ambos o acesso é público e não tem retorno de valor, portanto ser do tipo *Void*. Ambos recebem também como parâmetro um número real que passará para dentro do método com o nome de valor.

Métodos construtores

Os métodos construtores, ou simplesmente Construtores, são métodos especiais que são chamados automaticamente quando instâncias são criadas através da palavra-chave *new*. Por meio da criação de construtores, podemos garantir que o código que eles contêm será executado antes de qualquer outro código em outros métodos.

Os construtores são úteis para iniciar campos de instâncias de classes para garantir que, quando métodos dessas instâncias forem chamados eles contenham valores específicos. Caso os campos de uma instância não sejam iniciados, os seguintes valores são adotados:

- Campos do tipo boolean são iniciados automaticamente com a constante *false*;
- Campos do tipo char são iniciados com o código Unicode zero, que é impresso como um espaço (ou seja: " ");
- Campos de tipos inteiros (*byte*, *short*, *int*, *long*) ou de ponto flutuante (*float*, *double*) são automaticamente iniciados com o valor zero, do tipo do campo declarado.
- Instâncias de qualquer classe, inclusive da classe *String*, são iniciadas automaticamente com *null*.

As diferenças básicas entre construtores e outros métodos são:

- A palavra-chave *null* é usada em referências, indicando que as referências não foram iniciadas mas têm algum "valor" associado a elas, apesar desse "valor" não poder ser usado como uma instância poderia.
- Construtores devem ter exatamente o mesmo nome da classe a que pertencem, inclusive considerando maiúsculas e minúsculas;
- Construtores não podem retornar nenhum valor, nem mesmo *void*, portanto devem ser declarados sem tipo de retorno;
- Construtores não devem receber modificadores como *public* ou *private* e serão públicos se a classe for pública.

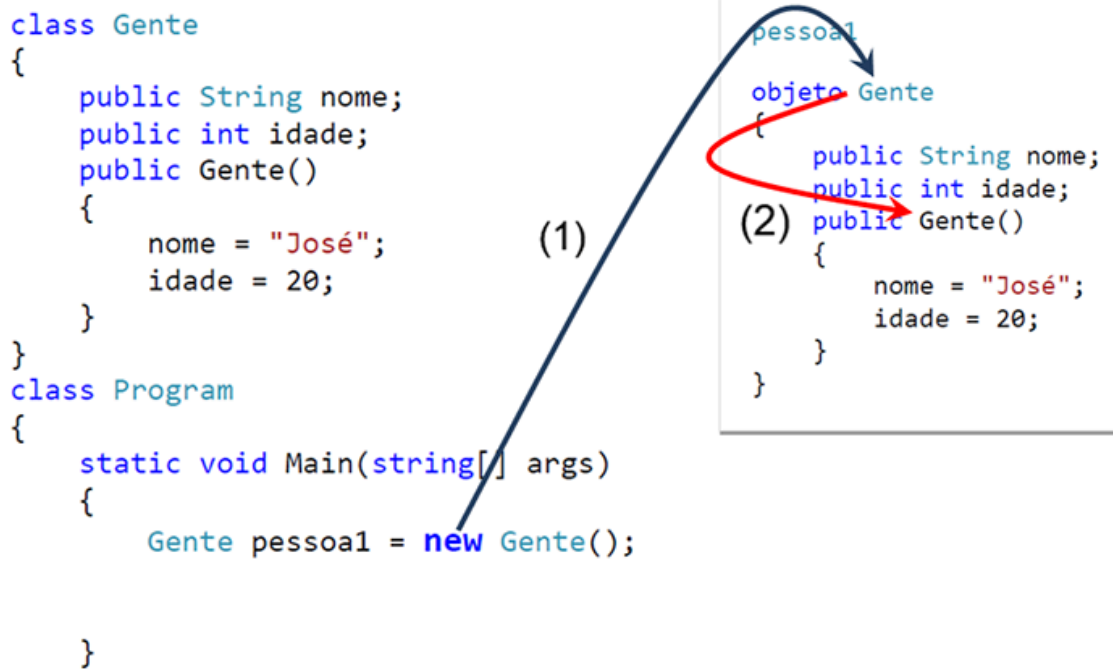
A razão pela qual os construtores têm regras mais rígidas para nomenclatura que os métodos é que, quando uma instância de uma classe que tem construtores for iniciada com a palavra-chave *new*, o compilador executará automaticamente o construtor, precisando então saber exatamente qual é o nome deste. Uma outra diferença significativa entre construtores e métodos comuns é que o programador não pode chamar construtores diretamente – somente quando a instância for iniciada com *new*.

O construtor é um método com mesmo nome que a classe. O programador de uma classe pode definir o construtor de uma classe, que é invocado toda vez que o programa instancia um objeto desta classe. (Deitel, 2003, p. 386)

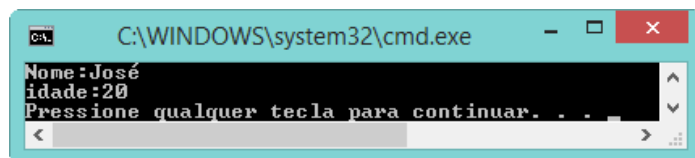
Assim temos a aplicação de um método construtor em uma classe no Código a seguir.

```
namespace Construtor01
{
    class Gente
    {
        public String nome;
        public int idade;
        public Gente()
        {
            nome = "José";
            idade = 20;
        }
    }
    class Program
    {
        static void Main(string[] args)
        {
            Gente pessoa1 = new Gente();
            Console.WriteLine("Nome:{0}\nidade{1}", pessoa1.nome, pessoa1.idade);
        }
    }
}
```

Acompanhando o processamento do programa , ao ser criado o objeto pessoa1 (1) pelo comando `new` imediatamente o sistema passa a executar o método construtor (2).



Desta forma a saída fica:



Sobrecarga de métodos

A possibilidade de criar mais de um método com o mesmo nome e assinaturas diferentes é conhecida como Sobrecarga de métodos. A decisão sobre qual método será chamado quando existem dois ou mais métodos será feita pelo compilador, baseado na assinatura dos métodos. (Santos, 2003, p. 77)

Linguagens orientadas a objeto permitem a criação de métodos com nomes iguais, contanto que as suas assinaturas sejam diferentes. . A assinatura de um método é composta de seu nome mais os tipos de argumentos que são passados para esse método, independentemente dos nomes de variáveis usadas na declaração do método. . Assim sendo no Código 29 temos a sobrecarga do método construtor da classe Gente. Como vemos cada um dos métodos com mesmos nomes possuem a composição de tipos diferentes como parâmetros em cada um deles.


```
class Gente
{
    public String nome;
    public int idade;
    public Gente()
    {
        nome = "José";
        idade = 20;
    }
    public Gente(String valor)
    {
        nome = valor;
        idade = 20;
    }
    public Gente(int valor)
    {
        nome = "José";
        idade = valor;
    }
    public Gente(String valorn, int valori)
    {
        nome = valorn;
        idade = valori;
    }
}
```

As assinaturas são:

```
public Gente()
{
    nome = "José";
    idade = 20;
}
```

assinatura Gente()

assinatura gente-vazia

```
public Gente(String valor)
{
    nome = valor;
    idade = 20;
}
```

assinatura Gente(String)

assinatura Gente-String

```
public Gente(int valor)
{
    nome = "José";
    idade = valor;
}
```

assinatura Gente(int)

Assinatura Gente-int

```
public Gente(String valorn, int valori)
{
    nome = valorn;
    idade = valori;
}
```

assinatura Gente(String int)

Assinatura Gente-String-int

Exemplo de chamada de Método construtor sobrecarregado:

```

public Gente(String valor)
{
    nome = valor;
    idade = 20;
}

class Program
{
    static void Main(string[] args)
    {
        Gente pessoa1 = new Gente("Maria");
        Console.WriteLine("Nome:{0}\nidade:{1}",
            pessoa1.nome, pessoa1.idade);
    }
}

```

A palavra-chave *this*

É criada internamente para cada instância, uma "auto referência", ou seja, uma referência à própria instância. Essa referência é representada pela palavra-chave *this*. Para chamar um construtor de dentro do outro, basta usar a palavra-chave *this* substituindo o nome do construtor. Construtores não podem ser chamados indiscriminadamente de dentro de qualquer método: existem algumas regras para a chamada de construtores que são:

- Somente construtores podem chamar construtores como sub-rotinas;
- Se um construtor for chamado a partir de outro, a chamada deve ser a primeira linha de código dentro do corpo do construtor;
- Construtores não são chamados pelos seus nomes, e sim por *this*;
- Construtores podem chamar outros métodos. Por exemplo, pode ser interessante ter um construtor e um método que iniciem as instâncias e chamar o método de dentro do construtor. Métodos não podem chamar construtores, nem mesmo com *this*.
- Construtores não podem ser chamados recursivamente: um construtor só pode chamar diretamente outro construtor e não a si próprio.

O uso mais imediato é como uma auto referência dos atributos dentro do objeto. Nele a palavra chave passa a se referir diretamente aos atributos da classe.

```

class Gente
{
    public String nome;
    public int idade;
    public Gente(String nome, int idade)
    {
        this.nome = nome;
        this.idade = idade;
    }
}

class Program
{
    static void Main(string[] args)
    {
        Gente pessoa1 = new Gente("Paulo",23);
    }
}

```

Uma analogia pode ser feita com o coringa do baralho. A palavra *this* passa a substituir a referência. Desta forma o atributo *nome*, passa a ser uma outra entidade diferente do parâmetro *nome* que é passado pela criação do objeto.

```

class Gente
{
    public String nome;
    public int idade;
    public Gente(String nome, int idade)
    {
        this.nome = nome;
        this.idade = idade;
    }
}

class Program
{
    static void Main(string[] args)
    {
        Gente pessoa1 = new Gente("Paulo", 23);
    }
}

```

The diagram illustrates the relationship between the `Gente` class and its instance `pessoa1`. It shows the `Gente` class with attributes `nome` and `idade`, and a constructor that initializes them. The `Program` class has a `Main` method that creates a new `Gente` object named `pessoa1` with the name "Paulo" and age 23. Two arrows point from the `pessoa1 = new Gente("Paulo", 23);` line in `Program` to the constructor and the attribute assignments in the `Gente` class, indicating the flow of data and object creation.

Integração entre classes

Encapsulamento

Encapsulamento é a característica da OO de ocultar partes independentes da implementação. O encapsulamento permite que você construa partes ocultas da implementação do *software*, que atingem uma funcionalidade e ocultam os detalhes de implementação do mundo exterior (SINTES, 2002, p. 22).

problemas de acessos indevidos é que se faz o encapsulamento.

O conceito de encapsulamento diz respeito ao uso de classes, através de seus métodos públicos, por outras classes. Este conceito nos diz como interligar classes onde existam dependências. Ele é o mecanismo utilizado para disponibilizar métodos, protegendo o acesso direto indevido aos atributos de uma instância (objeto). O encapsulamento evita a interferência externa indevida de um objeto sobre os dados de outros objetos a ele referenciados.

M4.2 Encapsulamento

Encapsulamento é a característica da OO de ocultar partes independentes da implementação. O encapsulamento permite que você construa partes ocultas da implementação do *software*, que atingem uma funcionalidade e ocultam os detalhes de implementação do mundo exterior (SINTES, 2002, p. 22).

O conceito de encapsulamento diz respeito ao uso de classes, através de seus métodos públicos, por outras classes. Este conceito nos diz como interligar classes onde existam dependências. Ele é o mecanismo utilizado para disponibilizar métodos, protegendo o acesso direto indevido aos atributos de uma instância (objeto). O encapsulamento evita a interferência externa indevida de um objeto sobre os dados de outros objetos a ele referenciados.

Modificadores de acesso

Uma das principais vantagens do paradigma da orientação a objetos é a possibilidade de encapsular os campos bem como os métodos capazes de manipular esses campos em uma classe. É desejável que os campos das classes fiquem ocultos ou escondidos dos programadores usuários dessas classes para evitar que os dados sejam manipulados diretamente, mas que sejam manipulados apenas por intermédio dos métodos da classe. A restrição ao acesso a atributos e métodos em classes por meio de modificadores de acesso que são declarados dentro das classes, antes dos métodos e dos campos. A restrição de acesso é estabelecida na definição da classe usando um dos modificadores: *private*, *protected*, *public* ou sem informação.

- Modificador *public*: garante que o atributo ou método da classe declarado com este modificador poderá ser acessado ou executado a partir de qualquer outra classe, ou seja sem restrição. Os atributos e métodos que devam ser acessados (e modificados, no caso de campos) devem ser declarados com este modificador.
- Modificador *private*: atributos ou métodos declarados com este modificador só podem ser acessados, modificados ou executados por métodos da própria classe, sendo completamente ocultos para o programador usuário que usar instâncias desta classe ou criar classes herdeiras ou

derivadas. Campos ou métodos que devam ser completamente ocultos de usuários da classe devem ser declarados com este modificador.

- Modificador *protected*: funciona como o modificador *private*, exceto que classes herdeiras ou derivadas também terão acesso ao campo ou método marcado com este modificador. Assim é permitido o acesso a todas as classes derivadas

As regras básicas para implementação de políticas para classes simples são:

- Todos os campos de uma classe devem ser declarados com o modificador *private* ou *protected*, ficando dessa forma ocultos para o programador usuário dessas classes.
- Métodos que devam ser acessíveis devem ser declarados explicitamente com o modificador *public*.
- Como a princípio os campos terão o modificador *private*, métodos que permitam a manipulação controlada dos valores dos campos (conhecidos por métodos "*getters*" e "*setters*" ou encapsulamento) devem ser escritos nas classes e estes métodos devem ter o modificador *public*.
- Na medida do necessário, os métodos de uma classe podem ser declarados com o modificador *private* – esses métodos não poderão ser executados por classes escritas por programadores usuários mas poderão ser executados por outros métodos dentro da mesma classe.

Classe Conta sem encapsulamento:

```
class Conta
{
    public String nome;
    public double saldo;
    public Conta(String nome)
    {
        this.nome = nome;
        this.saldo = 0;
    }
    public void depositar(double valor)
    {
        this.saldo += valor;
    }
    public void sacar(double valor)
    {
        this.saldo -= valor;
    }
}
```

Escondendo um atributo:

```
class Conta
{
    public String nome;
    private double saldo;
```

O programa apresenta erro:

```

class Program
{
    static void Main(string[] args)
    {
        Conta ccJF = new Conta("José Frederico");
        ccJF.depositar(500);
        ccJF.saldo = 1000000;
        ccJF.sacar(200000);
        Console.WriteLine("Nome:{0}\nSaldo:{1}",
            ccJF.nome, ccJF.saldo);
    }
}

```

double Conta.saldo

Error:

'NecessidadeEncapsular.Conta.saldo' is inaccessible due to its protection level

Para tornar visível externamente criamos um método (getSaldo()) para acessar ler apenas o conteúdo do atributo saldo

```

class Conta
{
    public String nome;
    private double saldo;
    public Conta(String nome)
    {
        this.nome = nome;
        this.saldo = 0;
    }
    public void depositar(double valor)
    {
        this.saldo += valor;
    }
    public void sacar(double valor)
    {
        this.saldo -= valor;
    }
    public double getSaldo()
    {
        return this.saldo;
    }
}

```

Assim programa principal passa a ter acesso apenas ao valor do saldo, mas não consegue alterar o seu valor.

```

class Program
{
    static void Main(string[] args)
    {
        Conta ccJF = new Conta("José Frederico");
        ccJF.depositar(500);
        // ccJF.saldo = 1000000;
        ccJF.sacar(200000);
        Console.WriteLine("Nome:{0}\nSaldo:{1}",
            ccJF.nome, ccJF.getSaldo());
    }
}

```

em uma classe é recomendável que todos os atributos fiquem *private* ou *protected*, Para um atributo poder ser alterado externamente mas sem acesso direto, monta-se um método para inserir um valor ao atributo . Para esta técnica se dá o nome de “*getters*” e “*setters*”.

```
private String nome;
public String getNome()
{
    return this.nome;
}
public void setNome(String nome)
{
    this.nome = nome;
}
```

técnica de encapsular por meio de getters e setters

O nome getNome e setNome é apenas uma convenção. O *get* indica que um valor será devolvido e o *set* indica que um valor será atribuído num atributo chamado nome. Assim sendo temos na Tabela a seguir, um resumo completo do processo de encapsulamento de um atributo público.

Sem encapsular	Encapsulamento
public Tipo atributo;	private Tipo atributo; public Tipo getAtributo() { return this.atributo; } public void setAtributo(Tipo valor) { this.atributo = valor; }
Referencia.atributo=x;	Referencia.setAtributo(x);
x=Referencia.atributo;	x=Referencia.getAtributo();

A forma geral aceita em todas as linguagens OO de encapsulamento é pelos métodos dos “*getters*” e “*setters*” porém a linguagem C# permite outras formas de encapsulamento. Uma delas é encapsular sem

transformar em método, mas criando uma variável de passagem .

```
class Conta
{
    private String nome;

    public String Nome
    {
        get { return nome; }
        set { nome = value; }
    }
}

class Program
{
    static void Main(string[] args)
    {
        Conta ccJF = new Conta();
        ccJF.Nome = "Jose Frederico";
        Console.WriteLine("Nome:{0}", ccJF.Nome);
    }
}
```

Assim o atributo nome fica protegido por uma variável de passagem Nome

- **Modificador Static**

Os modificadores *static* também são conhecidos como campos estáticos em classes. Os Campos estáticos de uma classe são compartilhados por todas as instâncias dessa classe, isto é, somente um valor será armazenado em um campo estático, e caso esse valor seja modificado por uma das instâncias da classe, a modificação será refletida em todas as outras instâncias dessa classe.

Campos estáticos são declarados com o modificador *static*, que deve ser declarado antes do tipo de dado do campo e pode ser combinado com modificadores de acesso como *public* e *private*. Campos estáticos também são conhecidos como campos de classes, já que esses campos poderão ser acessados diretamente usando o nome da classe, sem que sejam necessárias a criação de uma instância da classe e uma referência para tal instância.

Assim a palavra *static* tem como característica:

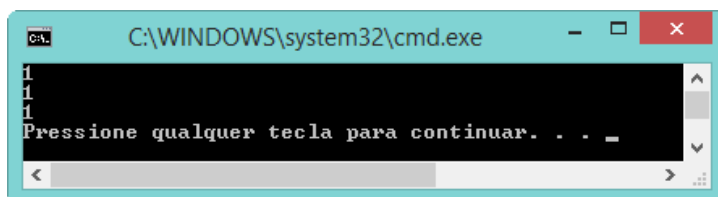
```
public class Exemplo
{
    private int seq = 0;
    public int nro;
    public Exemplo()
    {
        nro = ++seq;
    }
}

class Program
{
    static void Main(string[] args)
    {
        Exemplo e1 = new Exemplo();
        Console.WriteLine(e1.nro);

        Exemplo e2 = new Exemplo();
        Console.WriteLine(e2.nro);

        Exemplo e3 = new Exemplo();
        Console.WriteLine(e3.nro);
    }
}
```

Assim a cada instância criada o atributo seq inicia com o valor 0, assim cada uma das instâncias mostram o valor 1 contido no atributo nro



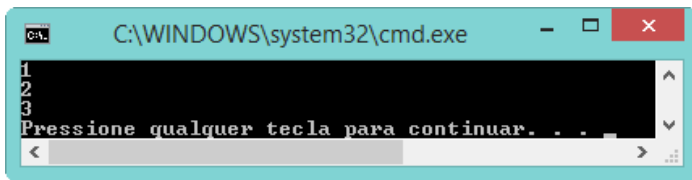
Alterando a classe Exemplo acrescentando o modificador *static* no atributo seq, e executar o mesmo programa

```
public class Exemplo
{
    private static int seq = 0;
    public int nro;
    public Exemplo()
    {
        nro = ++seq;
    }
}

class Program
{
    static void Main(string[] args)
    {
        Exemplo e1 = new Exemplo();
        Console.WriteLine(e1.nro);

        Exemplo e2 = new Exemplo();
        Console.WriteLine(e2.nro);

        Exemplo e3 = new Exemplo();
        Console.WriteLine(e3.nro);
    }
}
```


**Exercício 1:**

Um atributo também pode ser chamado de :

A)

Variável ou campo

B)

Característica ou tipo

C)

Valor

D)

Conjunto

E)

Parâmetro

Exercício 2:

B1-7 2

Um atributo serve para :

A)

indicar as possíveis informações armazenadas por um objeto de uma classe, representando o estado de cada objeto

B)

definir o tipo de classe

C)

identificar a característica de um objeto

D)

descrever um objeto

E)

descrever as características de uma classe ou objeto

Exercício 3:

B1-7 3

Métodos podem ser definidos como:

A)

procedimentos que formam os comportamentos e serviços oferecidos por objetos de uma classe

B)

comportamentos de uma classe

C)

características de operações de um objeto

D)

Ações executadas por uma classe, sempre utilizadas em tempo de execução

E)

Conjunto de funções que caracterizam alteram o funcionamento de uma instância.

Exercício 4:

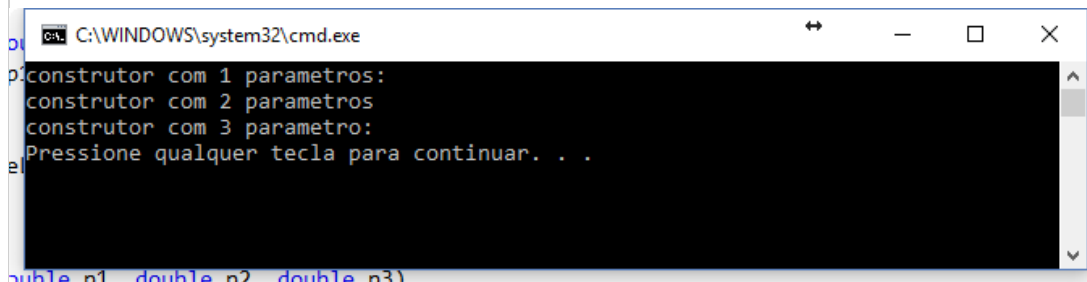
B1-7 4

Dado s código abaixo:

```
1 public class Classe1
2 {
3     double a1, a2, a3;
4
5     public Classe1(double p)
6         : this(p, p)
7     {
8         Console.WriteLine("construtor com 1 parametro: ");
9     }
10
11    public Classe1(double p12, double p3)
12        : this(p12, p12, p3)
13    {
14
15        Console.WriteLine("construtor com 2 parametros");
16    }
17
18    public Classe1(double p1, double p2, double p3)
19    {
20        this.a1 = p1;
21        this.a2 = p2;
22        this.a3 = p3;
23        Console.WriteLine("construtor com 3 parametros: ");
24    }
25 }
26 class Program
27 {
28     static void Main(string[] args)
29     {
30         Classe1 c1 = new Classe1(1);
31     }
32 }
```

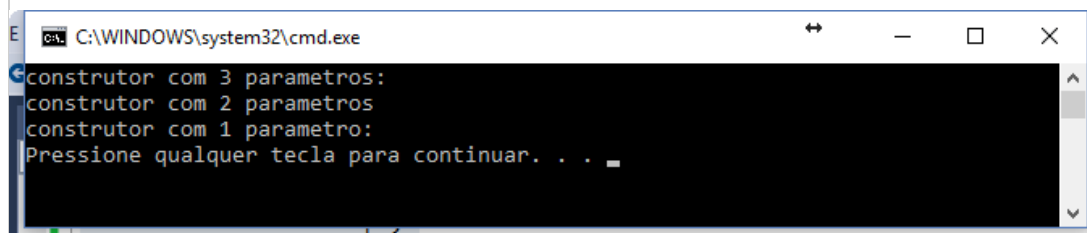
Qual a saída, e a sequência de execução correta?

A)



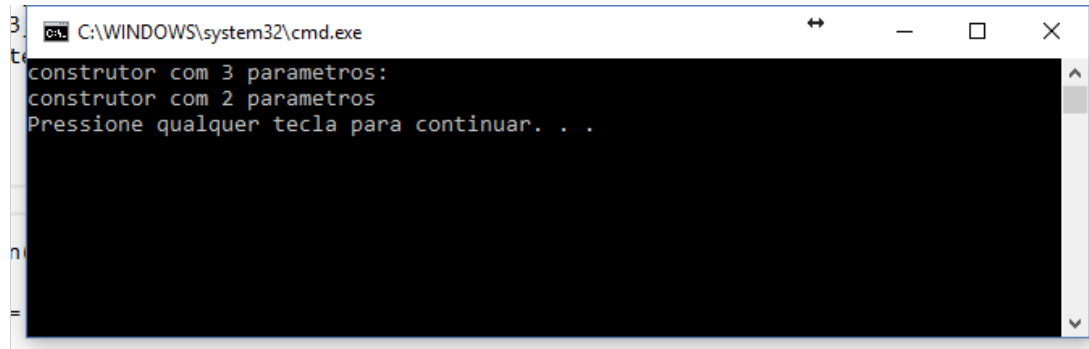
30,c1-5, c1-6,c1-8,c1-11,c1-12,c1-15,c1-18,c1-20,c1-21,c1-22,c1-23

B)



30,c1-5, c1-6,c1-11,c1-12,c1-18,c1-20,c1-21,c1-22,c1-23,c1-15,c1-8

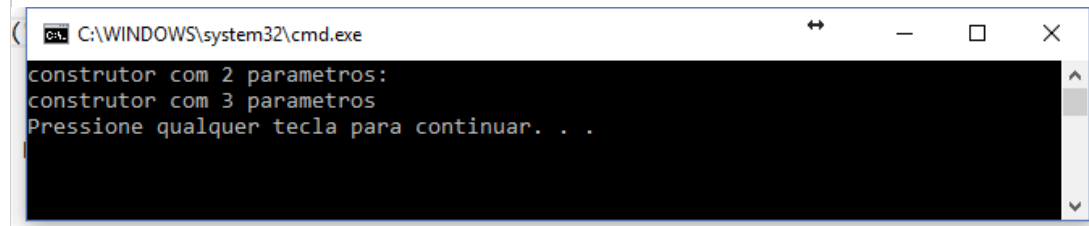
C)



```
C:\WINDOWS\system32\cmd.exe
construtor com 3 parametros:
construtor com 2 parametros
Pressione qualquer tecla para continuar. . .
```

30,c1-11,c1-12,c1-18,c1-20,c1-21,c1-22,c1-23,c1-15

D)



```
C:\WINDOWS\system32\cmd.exe
construtor com 2 parametros:
construtor com 3 parametros
Pressione qualquer tecla para continuar. . .
```

30,c1-11,c1-12,c1-15,c1-18,c1-20,c1-21,c1-22,c1-22

E)

Não compila, Erro.

Exercício 5:

B1-7 5

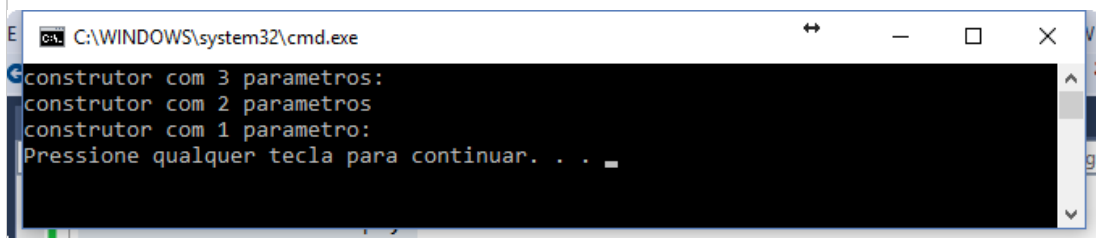
Dado o código abaixo

```
1  public class Classe1
2  {
3      double a1, a2, a3;
4
5      public Classe1(double p)
6          : this(p, p)
7      {
8          Console.WriteLine("construtor com 1 parametro: ");
9      }
10
11     public Classe1(double p12, double p3)
12         : this(p12, p12, p3)
13     {
14
15         Console.WriteLine("construtor com 2 parametros");
16     }
17
18     public Classe1(double p1, double p2, double p3)
19     {
20         this.a1 = p1;
21         this.a2 = p2;
22         this.a3 = p3;
23         Console.WriteLine("construtor com 3 parametros: ");
24     }
25 }
26 class Program
27 {
28     static void Main(string[] args)
29     {
30         Classe1 c1 = new Classe1();
31     }
32 }
```

O que acontece quando executamos o programa.

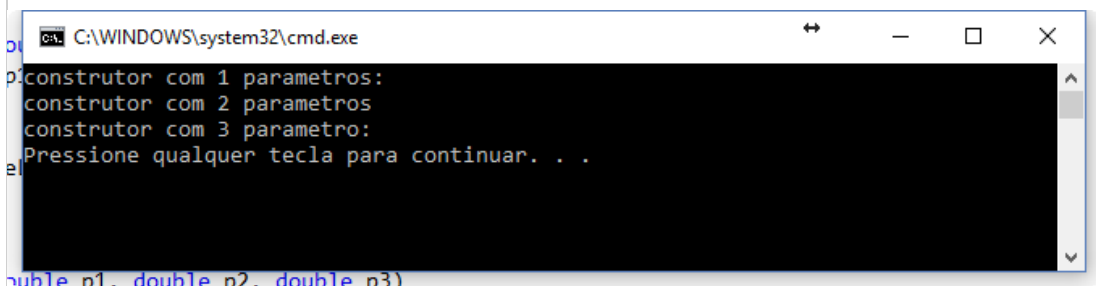
A)

Executa e apresenta a seguinte tela:



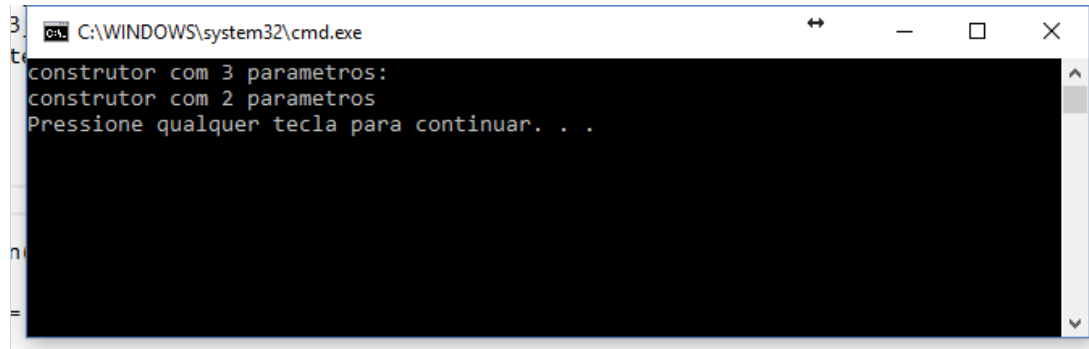
B)

Executa e apresenta a seguinte tela:



C)

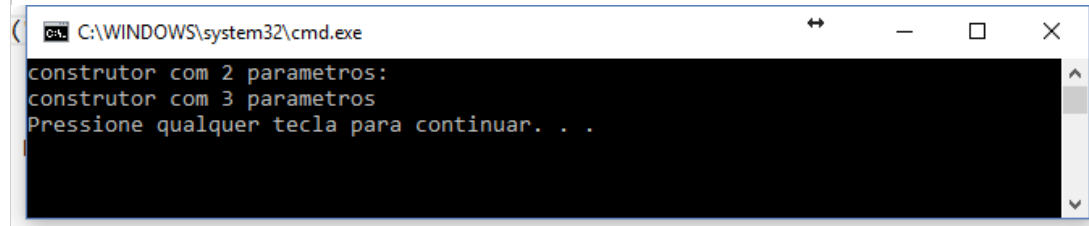
Executa e apresenta a seguinte tela:



```
C:\WINDOWS\system32\cmd.exe
construtor com 3 parametros:
construtor com 2 parametros
Pressione qualquer tecla para continuar. . .
```

D)

Executa e apresenta a seguinte tela:



```
C:\WINDOWS\system32\cmd.exe
construtor com 2 parametros:
construtor com 3 parametros
Pressione qualquer tecla para continuar. . .
```

E)

O programa tem um erro, não compila.

Exercício 6:

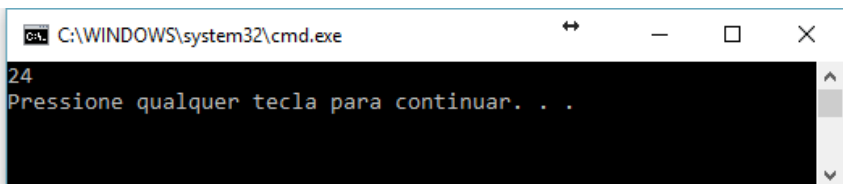
B1-7 7

Considere o código abaixo:

```
1  public class Classe1
2  {
3      public float d1;
4
5      public Classe1(float p)
6      {
7          this.d1=p;
8      }
9      public void metodo1(float m, float n)
10     {
11         d1 *= (m + n);
12     }
13     public void metodo1(float m)
14     {
15         this.metodo1(m, m);
16         d1 *= m;
17     }
18     public void metodo1()
19     {
20         this.metodo1(this.d1);
21     }
22 }
23 class Program
24 {
25     static void Main(string[] args)
26     {
27         Classe1 c1 = new Classe1(3);
28         c1.metodo1(1,1);
29         Console.WriteLine(c1.d1);
30     }
31 }
```

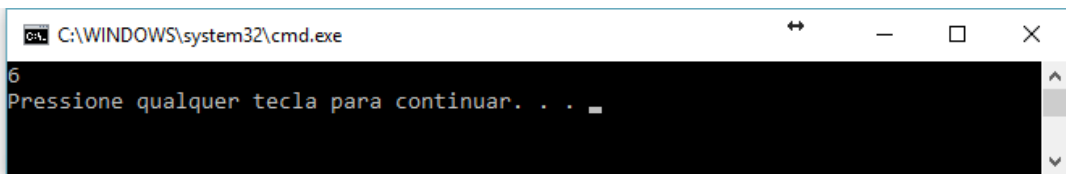
Qual a saída e a sequência de execução?

A)



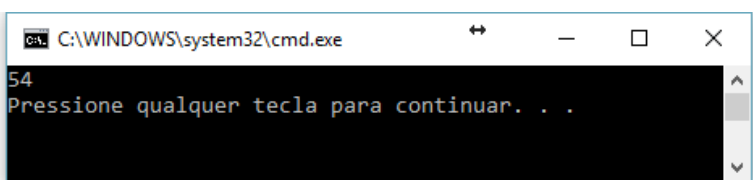
27, c1-5, c1-7, 28, c1-13, c1-15, c1-9, c1-11, c1-16, 29

B)



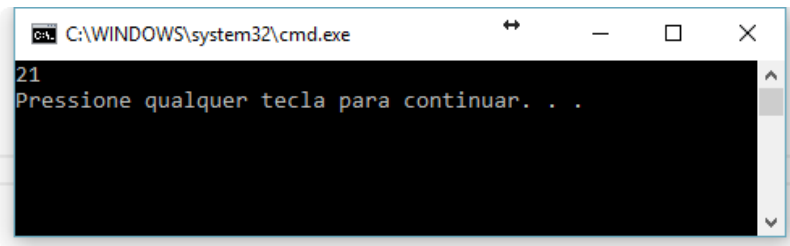
27, c1-5, c1-7, 28, c1-9, c1-11, 29

C)



27, c1-5, c1-7, 28, c1-18, c1-20, c1-13, c1-15, c1-9, c1-11, c1-16, 29

D)



27, c1-5, c1-7, 28, c1-18, c1-20, c1-13, c1-15, c1-9, c1-11, c1-16, 29

E)

erro, o programa não compila.

Exercício 7:

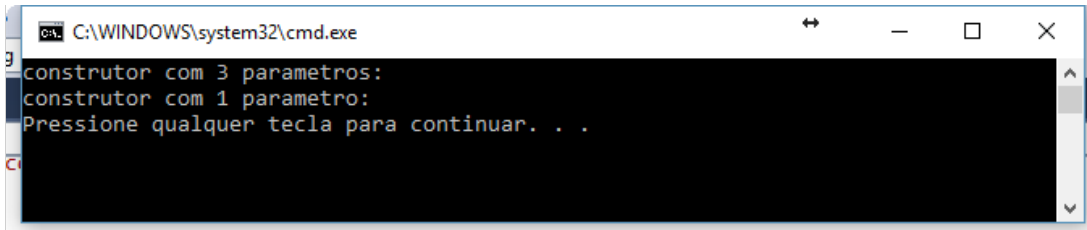
B1-7 6

Considere o código abaixo:

```
1      public class Classe1
2      {
3          double a1, a2, a3;
4
5          public Classe1(double p)
6              : this(p, p, p)
7          {
8              Console.WriteLine("construtor com 1 parametro: ");
9          }
10
11         public Classe1(double p12, double p3)
12             : this(p12, p12, p3)
13         {
14
15             Console.WriteLine("construtor com 2 parametros");
16         }
17
18         public Classe1(double p1, double p2, double p3)
19         {
20             this.a1 = p1;
21             this.a2 = p2;
22             this.a3 = p3;
23             Console.WriteLine("construtor com 3 parametros: ");
24         }
25     }
26     class Program
27     {
28         static void Main(string[] args)
29         {
30             Classe1 c1 = new Classe1(1,1);
31         }
32     }
33
```

ssinale a alternativa que contém a saída e a sequência de execução correta.

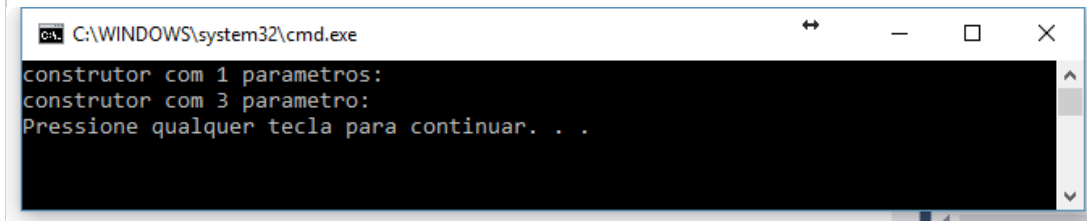
A)



```
C:\WINDOWS\system32\cmd.exe
construtor com 3 parametros:
construtor com 1 parametro:
Pressione qualquer tecla para continuar. . .
```

30,c1-5, c1-6,c1-18,c1-20,c1-21,c1-22,c1-23,c1-8

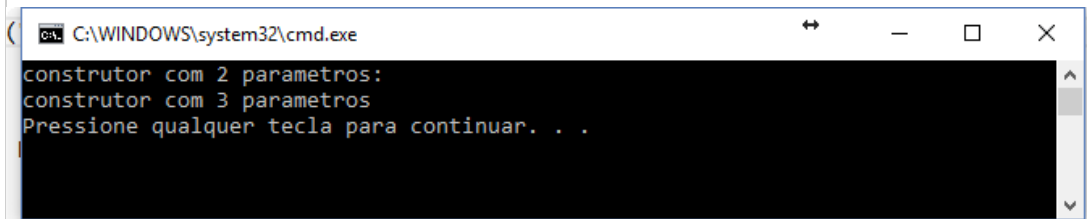
B)



```
C:\WINDOWS\system32\cmd.exe
construtor com 1 parametros:
construtor com 3 parametro:
Pressione qualquer tecla para continuar. . .
```

30,c1-5, c1-6 c1-8,c1-18,c1-20,c1-21,c1-22,c1-23

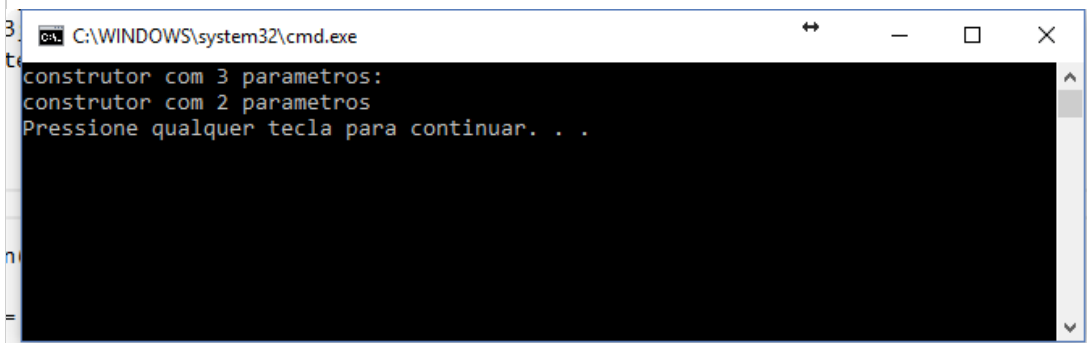
C)



```
C:\WINDOWS\system32\cmd.exe
construtor com 2 parametros:
construtor com 3 parametros
Pressione qualquer tecla para continuar. . .
```

30,c1-11,c1-12,c1-15,c1-18,c1-20,c1-21,c1-22,c1-22

D)



```
C:\WINDOWS\system32\cmd.exe
construtor com 3 parametros:
construtor com 2 parametros
Pressione qualquer tecla para continuar. . .
```

30,c1-11,c1-12,c1-18,c1-20,c1-21,c1-22,c1-23,c1-15

E)

Erro, o programa não compila.